Dell Global Operations

Data Science Case Study

Kit Cycle Time Prediction

Findings, Conclusions & Strategic Recommendations

Naveena Kondraganti

March 202

1. Introduction & Approach

1.1 Problem Context

In Dell's manufacturing pipeline, the kit phase involves gathering all materials necessary to build a product — from sending the order to a warehouse to bringing together every component. The duration of this phase (kit cycle time) directly impacts production scheduling, delivery commitments, and customer satisfaction.

This report documents the end-to-end data science process for predicting kit cycle time: from exploring 5 interconnected datasets to uncovering hidden data nuances, engineering predictive features, training explainable models, and deriving actionable business insights.

1.2 Approach

The problem was framed as a regression task: predict kit_cycle_days = (kit_end – kit_start) as a continuous value, round to the nearest integer, then compute kit_end by stepping forward through working days only. Regression was chosen over classification because the target is ordinal — a 1-day prediction error is fundamentally different from a 5-day error, which MAE captures but classification accuracy does not.

1.3 Evaluation Philosophy

Given the heavily concentrated target (48.6% of orders = 1 day), raw metric numbers are less meaningful than understanding where and why the model succeeds or fails. This report, therefore emphasizes:

- Process rigor — how data quality issues were discovered and handled
- Feature insight — which signals drive predictions and what they mean for business
- Error analysis — where the model struggles and what data could help
- Explainability(XAI) — SHAP-based decomposition of every prediction into understandable components

2. Exploratory Data Analysis

2.1 Dataset Landscape

Dataset	Rows	Columns	Join Key	Role
train.csv	179,853	19	Primary	Historical orders with kit_end
inference.csv	12,069	18	Primary	Orders needing prediction
calendar.csv	370	17	date	Fiscal calendar, weekends, holidays
capacity.csv	191,934	4	date	Factory hours (1 row per order)
expected_minutes .csv	260	4	family_desc	Product complexity metrics

2.2 Target Variable Analysis

The target (kit_cycle_days) is extremely concentrated: nearly half of all orders complete in exactly 1 day. This makes the problem inherently difficult — a model that simply predicts 1 for every order already achieves a surprisingly low error. The real challenge is correctly identifying the 0-day (same-day) and 3+ day orders.

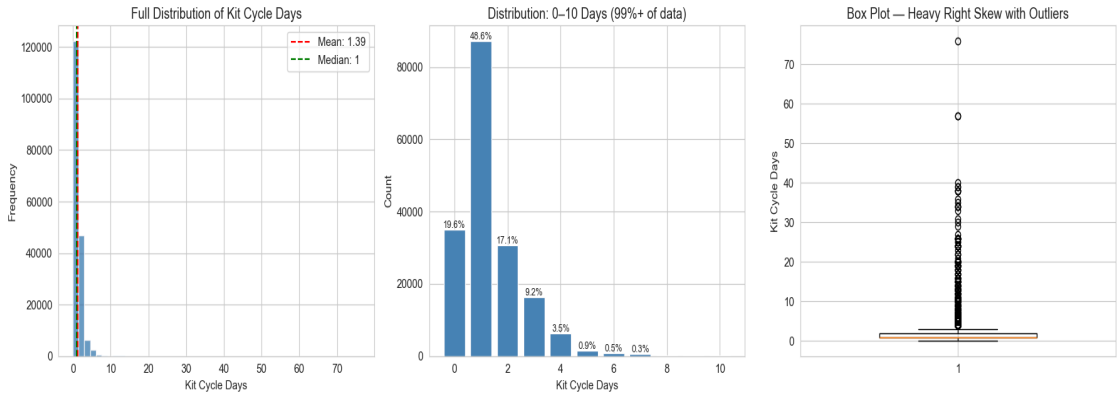

Figure 1: Target distribution — 48.6% of orders complete in 1 day, with a long right tail

Days	%	Cumulative	Business Meaning
0 (same day)	19.6%	19.6%	Express/small orders completed within the shift
1 day	48.6%	68.2%	Standard overnight kitting — the dominant pattern
2 days	17.1%	85.3%	Weekend span or complex product
3+ days	14.7%	100%	Large orders, capacity bottlenecks, or holidays

2.3 Univariate Analysis

2.3.1 Numerical Features

All three numerical features exhibit extreme right-skew (order_amt skew > 20). This confirmed the need for log-transformations. The 2,049 zero-amount orders (1.1%) were retained with a binary flag rather than dropped, as they represent legitimate internal transfers.

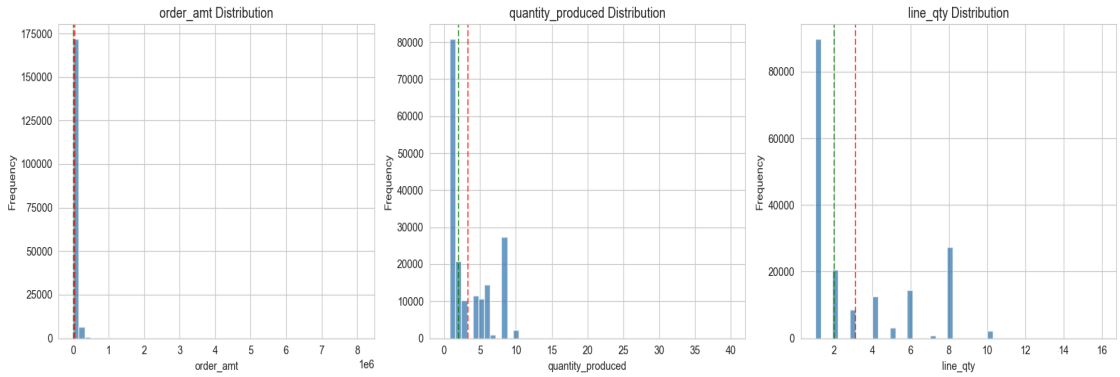

Figure 2: Numerical feature distributions — heavy right-skew requiring log-transforms

2.3.2 Categorical Features

Significant class imbalance across categoricals: buid has 85% in one class, mcid has 8 warehouses with highly uneven distribution. This informed the choice of smoothed target encoding over one-hot (which would create 360+ sparse columns for family_desc alone).

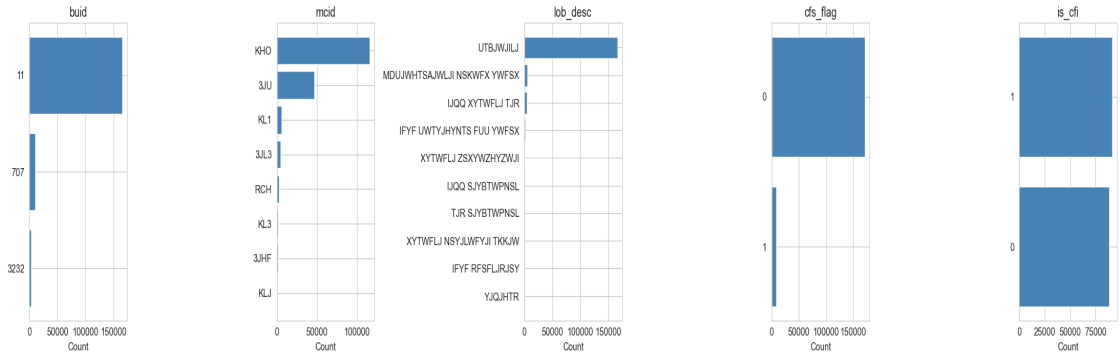

Figure 3: Categorical feature distributions — class imbalance across all features

2.3.3 Expected Manufacturing Minutes

Expected kit/build/burn minutes follow distinct distributions. Only 46% of training families have this data, requiring careful hierarchical imputation (family_parent → LOB → global median) to avoid information loss.

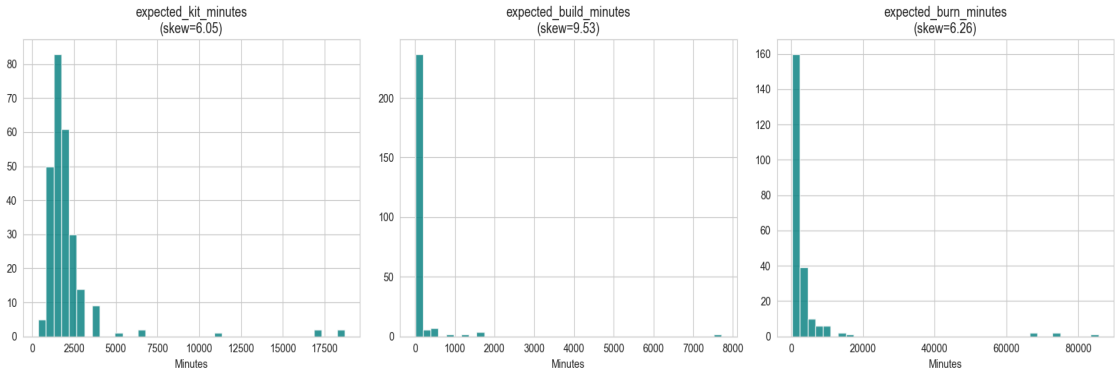

Figure 4: Expected minutes distributions — 46% coverage, hierarchical imputation needed

2.4 Bivariate Analysis & Causation

2.4.1 Day-of-Week: The Strongest Causal Signal

The day-of-week pattern is not merely correlational — there is a clear causal mechanism: the factory does not run manufacturing on weekends unless overtime is scheduled. A Saturday start must wait until Monday (or overtime Saturday) to complete, mechanically adding 1–2 days. Friday starts similarly bridge into weekends.

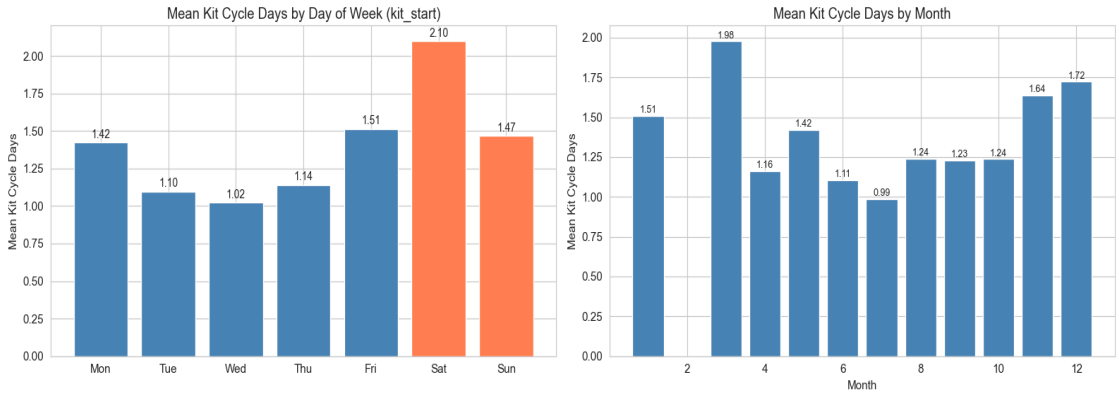

Figure 5: Kit cycle days by day-of-week — causal link via weekend factory shutdown

Day	Mean Cycle	Mechanism
Saturday	2.23 days	Must wait through Sunday (+ Saturday if no OT)
Friday	1.88 days	Bridges into weekend — completes Monday
Monday	1.07 days	Full work week ahead
Tue–Thu	1.30–1.40 days	Standard processing with no weekend gap

2.4.2 Categorical Features vs Target

Warehouse (mcid) and product family show meaningful variation. Warehouse KL3 averages ~2.0 days vs RCH at ~1.2 days, suggesting different operational characteristics or product mixes. The is_cfi flag (software customization) adds ~0.2 days on average.

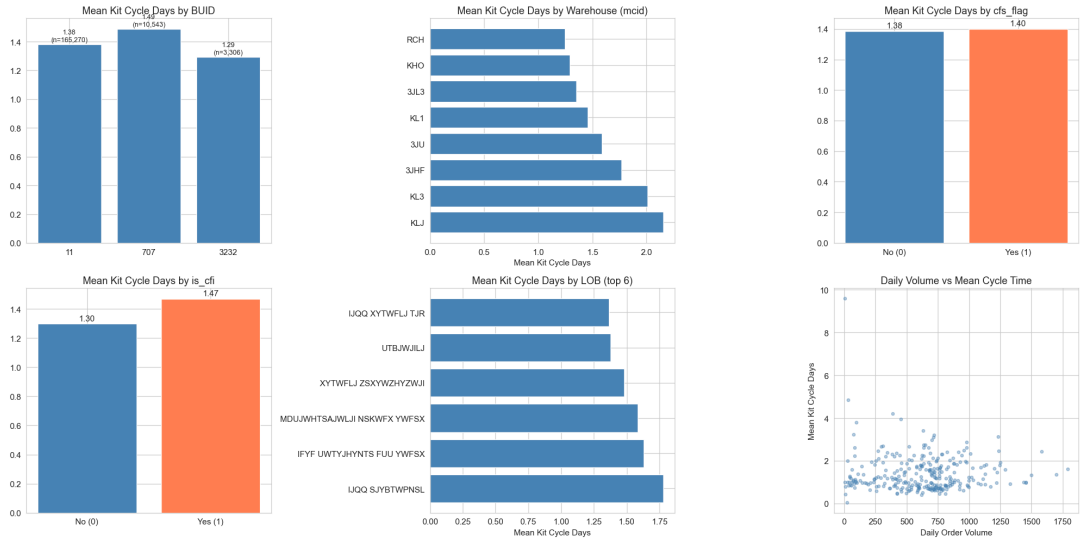

Figure 6: Cycle time by warehouse, business unit, LOB, and product flags

2.4.3 Correlation & Collinearity

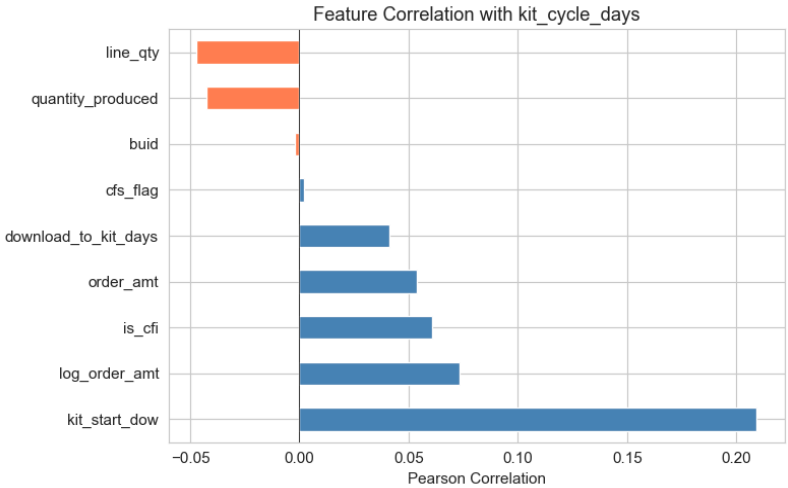

Figure 7: Correlation matrix — weak linear correlation with target, strong inter-feature collinearity

No single numeric feature has strong linear correlation with the target (all $|r| < 0.1$). The relationship is non-linear and interaction-driven. However, the feature-to-feature correlations expose critical collinearity — a 62×62 matrix analysis revealed 31 features with $|r| > 0.85$, addressed in Section 4.3.

2.5 Critical Data Nuances Discovered

Finding	Impact	Action
Capacity rows = individual orders, NOT duplicates	Dropping "duplicates" destroys daily_volume signal (r=1.0 with order count)	Aggregated by date
IDC weeks are 7 & 8 in data but 6 & 7 in docs	Documentation-data mismatch	Used data as-is (Assumption A1)
Calendar site="FCJ" vs Train site="KHO"	Cross-dataset join broken by site mismatch	Same factory, masked data — dropped site column
sum(line_qty) ≠ quantity_produced for some orders	Internal data inconsistency	Updated quantity_produced to sum(line_qty)
Weekend regular_hours should be 0	Misclassified capacity hours	Moved regular_hours → ot_hours on weekends/holidays
28 est_ship_dates = 1900-01-01	Placeholder values, not real dates	Flagged + imputed

2.6 Data Quality: Impossible Records

734 rows (0.4%) have kit_end before kit_start — physically impossible in manufacturing. Their clustered pattern suggests batch data entry errors rather than systematic issues.

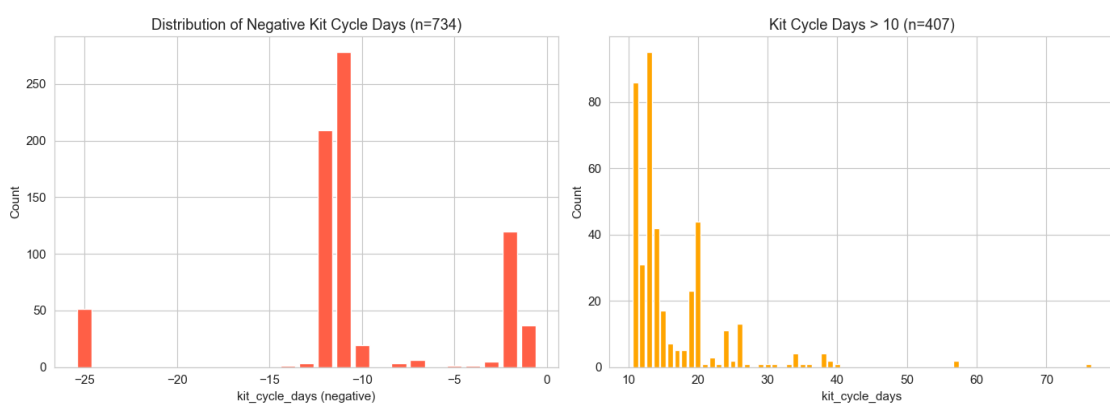

Figure 8: Negative cycle times — clustered pattern suggests batch data entry errors

3. Data Wrangling & Outlier Handling

3.1 Missing Value Strategy

Column	Missing	Strategy	Rationale
download_date	5 rows	Impute: kit_start – 1 day	Median lead time is ~1 day
est_ship_date	28 rows	Flag + impute: kit_start + 30	Preserve missingness as a signal
expected_burn_minutes	31/260 families	Hierarchical: parent → LOB → global median	Respect product hierarchy
expected_kit/build	46% null after merge	Same hierarchical approach	Avoid dropping half the data
capacity_date	12 rows	Drop	< 0.01%, negligible impact

3.2 Outlier Handling

- **Negative cycle times (734 rows):** Dropped — kit_end before kit_start is physically impossible.
- **OT hours > 8 (1 record):** Capped at 8 — total exceeded 24h/day, clearly erroneous.
- **Zero-amount orders (2,049):** Retained with flag — legitimate internal transfers/replacements.
- **quantity_produced mismatch:** Corrected to match sum(line_qty) for inconsistent orders.

3.3 Weekend/Holiday Capacity Correction

The factory only runs overtime on non-working days, but the raw data had regular_hours > 0 on weekends/holidays. We reclassified: regular_hours on non-working days were absorbed into ot_hours (regular set to 0). This ensures capacity_minutes correctly reflects actual production time.

4. Feature Engineering

4.1 Raw Features vs Engineered Features

The raw dataset has 19 columns — mostly identifiers and dates with minimal direct predictive value. Through systematic feature engineering, we created 62 features across 9 categories:

Category	Count	Key Features	Signal Type
Temporal	7	kit_start_dow, is_friday, days_to_weekend	Calendar pattern
Calendar	6	is_weekend, is_holiday, is_eoq, fiscal_qtr_weeknum	Factory schedule
Lead Time	4	download_to_kit_days, days_until_ship, is_urgent	Order urgency
Order Complexity	10	line_qty, log_order_amt, is_cfi, cfs_flag	Order size/type
Capacity	8	daily_volume, capacity_minutes, ot_ratio	Factory load
Product Complexity	9	log_expected_kit/build/burn, kit_to_capacity_ratio	Product difficulty
Target Encoding	4	family_desc_te, mcid_te, lob_te, parent_te	Category identity
Rolling/Lag	6	prev_cycle, rolling_mean, rolling_std, trend	Recent history
Volume Lags	3	vol_lag_1, vol_rolling_3d, vol_rolling_7d	Factory trend

4.2 Encoding: Smoothed Target Encoding

For high-cardinality categoricals (family_desc: 360 values), one-hot encoding creates sparse, high-dimensional features that hurt tree models. We used Leave-One-Out (LOO) target encoding with smoothing:

- **LOO formula:** te = (group_sum – this_row) / (group_count – 1) — prevents the current row from leaking into its own encoding.
- **Smoothing (α=20):** Blends toward the global mean for small groups. A family with 5 orders gets 80% global mean influence; a family with 500 orders gets 96% group-specific.
- **Time-awareness:** Encoding maps computed only on the training split, applied separately to validation and inference.

4.3 Collinearity Pruning: 62 → 14 Features

The 62×62 correlation matrix exposed massive redundancy. We systematically eliminated features with |r| > 0.85:

Collinear Pair	|r|	Kept	Dropped
total_hours ↔ capacity_minutes	1.00	capacity_minutes	total_hours

is_weekend_start ↔ is_weekend	1.00	is_weekend	is_weekend_start
est_kit_days ↔ kit_to_capacity_ratio	1.00	kit_to_capacity_ratio	est_kit_days
fiscal_month ↔ kit_start_month	0.996	kit_start_month	fiscal_month + 2 more
quantity_produced ↔ line_qty	0.93	line_qty	quantity_produced
effective_regular_hours ↔ is_weekend	–0.98	is_weekend	effective_regular_hours
rolling_mean_3 ↔ rolling_mean_7	0.90	rolling_mean	rolling_mean_3

Result: 14 features with zero redundancy, covering every independent signal axis. Fewer features = less noise, faster training, better generalization.

4.4 Interaction Terms & Derived Features

- **kit_to_capacity_ratio:** Product kit minutes/factory capacity — captures whether a product saturates available time.
- **consec_nw:** Consecutive non-working days after kit_start — explains why Saturday = 2 days (Sat + Sun both off).
- **rolling_trend:** prev_cycle – rolling_mean — captures momentum. A family trending upward signals worsening conditions.
- **dl_to_kit_is_1:** Binary flag for download_to_kit = 1 day — these fast-tracked orders have 25% zero-day completion rate.

4.5 Configurable Rolling Windows

All rolling/lag window sizes are parameterized in a single configuration cell, enabling rapid experimentation without touching function code or feature lists:

Parameter	Default	Tested	Effect
ROLLING_SHORT	2	2, 3, 5	Short-term family mean
ROLLING_LONG	7	5, 7, 10, 14	Medium-term family mean
VOL_LAG	1	1, 2, 3	Volume lookback days

5. Model Training & Hyperparameter Tuning

5.1 Validation Strategy

A strict time-based split prevents future-information leakage: Train = March–November 2025 (145,742 orders), Validation = December 2025–January 2026 (33,377 orders). This simulates the real deployment scenario.

5.2 Bayesian Optimization with Optuna

Each of the three gradient boosting models underwent 50 trials of Bayesian hyperparameter optimization using Optuna's TPE sampler. Key search dimensions:

Parameter	Search Range	Best (LightGBM)	Best (XGBoost)
learning_rate	0.01 – 0.1 (log-scale)	0.019	0.096
max_depth	4 – 10	8	10
num_leaves	20 – 127	42	—
min_child_samples	20 – 100	62	23
subsample	0.6 – 1.0	0.63	0.74
colsample_bytree	0.5 – 1.0	0.50	0.72
reg_alpha / reg_lambda	0.001 – 5.0 (log)	0.042 / 0.645	0.033 / 0.205
n_estimators	500 – 3,000	1,137	2,466

A weighted ensemble (inverse-MAE weights) combines all three models' predictions. LightGBM and XGBoost performed equally; CatBoost was slightly behind. The ensemble provides marginal stability over any single model.

5.3 Error Analysis by Segment

Rather than focusing on a single aggregate metric, we analyzed where the model succeeds and fails:

Segment	% of Data	Difficulty	Observation
0-day orders	19.6%	Hard	The model tends to predict 1 instead of 0 — minority class under-represented
1-day orders	48.6%	Easy	Dominant pattern — model captures this well
2-day orders	17.1%	Medium	Often confused with 1-day or 3-day boundaries
3+ day orders	14.7%	Hardest	Rare events with high variance — insufficient signal in current features
Saturday starts	~5%	Medium	Improved by weekend features, but OT variability adds noise
New families	<1%	Medium	Falls back to the global mean via encoding smoothing

The error analysis reveals the remaining prediction gap is primarily data-limited, not model-limited. The 0-day and 3+ day segments need richer signals (material availability, WIP counts) that aren't in the current dataset.

6. Explainable AI: SHAP Analysis

6.1 Global Feature Importance

SHAP (SHapley Additive exPlanations) decomposes each prediction into per-feature contributions. This makes the model's reasoning transparent and auditable — critical for production deployment trust.

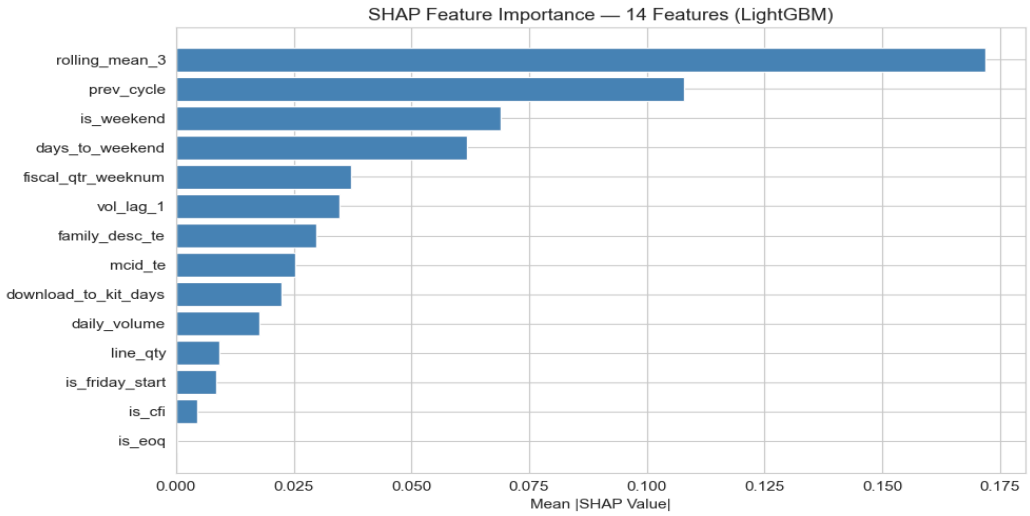

Figure 9: Global SHAP importance — rolling features dominate, followed by DOW and product identity

6.2 Top Predictors & Interpretation

Rank	Feature	SHAP Direction	Business Interpretation
1	prev_cycle	Strong positive	If a family just took 3 days, it will likely take 3 days again
2	rolling_mean	Strong positive	Sustained trend — rising mean signals worsening conditions for that family
3	family_desc_te	Both directions	Product identity — some families are inherently fast, others slow
4	days_to_weekend	Negative (low value = longer)	Closer to Saturday → higher prediction (weekend gap effect)
5	is_friday_start	Positive when 1	Friday starts push prediction up ~0.3 days on average
6	daily_volume	Positive	Factory congestion: more orders → longer cycles
7	mcid_te	Both directions	Warehouse-specific: KL3 pushes up, RCH pushes down

6.3 SHAP Beeswarm: Feature Value → Prediction Impact

Each dot is one prediction. Red = high feature value, blue = low. Dots pushed right increase the predicted cycle time.

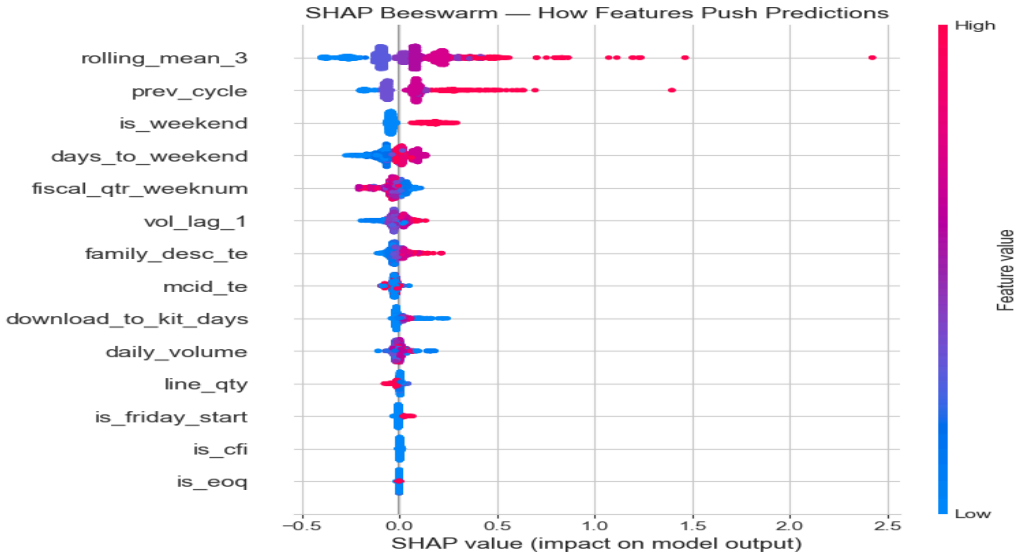

Figure 10: SHAP beeswarm — high prev_cycle (red) consistently pushes predictions right (longer)

6.4 Cross-Model Consistency

SHAP computed across all three model architectures shows consistent feature rankings, confirming the signals are data-driven, not model artifacts:

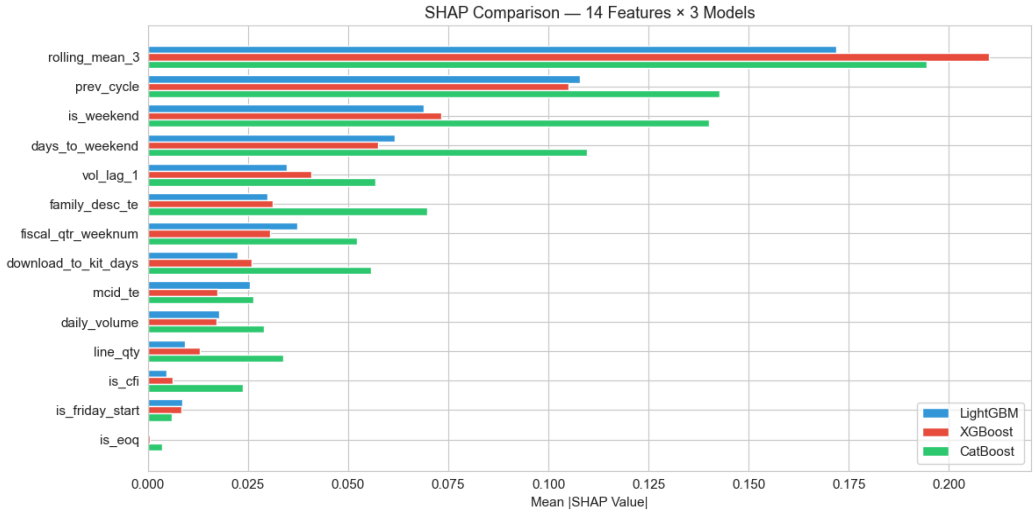

Figure 11: SHAP comparison — all three models agree on top predictors

7. Assumptions

#	Observation	Assumption & Action
A1	is_idc=1 for fiscal weeks 7 & 8, but case study says 6 & 7	Data quality issue — used data as-is, IDC = weeks 7 & 8
A2	Calendar site = "FCJ", Train site = "KHO"	Same factory, masked data — dropped site, joined on date
A3	site and alt_type_desc are constant (zero variance)	Removed both — no predictive information
A4	sum(line_qty) ≠ quantity_produced for some orders	Updated quantity_produced to sum(line_qty) for those orders
A5	Weekends/holidays: only OT, not regular shifts	Moved regular_hours → ot_hours on non-working days
A6	734 rows: kit_end < kit_start (impossible)	Dropped — kit_end cannot precede kit_start
A7	28 est_ship_dates = 1900-01-01 (placeholder)	Flagged + imputed with kit_start + 30 days
A8	Capacity rows look like duplicates	Each row = one order — aggregated by date (confirmed r=1.0)
A9	kit_end computation for inference	Calendar+capacity-aware: day counts if total_hours > 0, even on weekends

8. Business Insights & Recommendations

8.1 Operational Findings

- **Friday/Saturday scheduling penalty:** 40–65% longer cycle times vs mid-week starts. Shifting non-urgent kits to Tue–Thu could save 0.5–1.0 days per order at zero cost.
- **Warehouse performance gap:** KL3 averages ~2.0 days vs RCH at ~1.2 days. An operational audit of KL3 could reveal process bottlenecks.
- **Volume-driven congestion:** Above ~810 orders/day (75th percentile), cycle time rises. A real-time volume threshold could trigger proactive overtime.
- **End-of-quarter surge:** Fiscal weeks 12–13 show elevated cycles. Pre-positioning materials before quarter-end would mitigate.
- **Product family predictability:** Some families are 74% same-day, others average 2+ days. Per-family SLA commitments would be more accurate than blanket estimates.
- **Software customization impact:** is_cfi=1 adds ~0.2 days. Parallel software tracks could reduce total kit time.

8.2 Data-Centric Improvements

The remaining prediction gap is data-limited. Additional signals that would improve accuracy:

- If we have the Kit_start and Kit_end with timestamps, we can predict more accurately
- Supplier lead time data — late materials are a key cause of extended kitting
- Workforce scheduling — actual staffing, not just available hours
- Historical rework/quality hold data — holds extend cycle time unpredictably
- Material availability flags — whether all components were in stock at kit_start
- Order priority/expedite flags — rush orders may skip queues

8.3 Model-Centric Improvements

- **Classification-regression hybrid:** A classifier for 0-day detection (hardest class), then a regressor for 1+ days. Combines the strengths of both approaches.
- **Quantile regression:** Predict 10th/50th/90th percentiles for best/expected/worst case — more actionable than point estimates.
- **Temporal cross-validation:** Expanding-window CV instead of a single split for more robust parameter selection.
- **Neural network augmentation:** LSTM/Transformer on order sequences could capture temporal patterns that tree models miss.

9. Production Readiness & Next Steps

Moving from a Jupyter notebook to a production prediction system requires the following infrastructure:

9.1 MLflow: Experiment Tracking & Model Registry

- Track all Optuna trials, hyperparameters, metrics, and model artifacts
- Register the best model with staging → production lifecycle management
- Version control models alongside feature configurations for reproducibility
- Enable A/B testing between model versions before promotion

9.2 Feature Store (AWS SageMaker Feature Store, Feast)

- Centralize feature definitions: rolling_mean, vol_lag, target encodings, capacity metrics
- Point-in-time correct feature serving (prevent train/serve skew)
- Real-time feature updates: refresh prev_cycle as each order completes
- Cross-team feature sharing (planning, logistics, customer ETA)

9.3 Data & Model Drift Monitoring (Evidently AI)

- Monitor input feature distributions for covariate shift (PSI thresholds)
- Track prediction distribution — alert if 3+ day predictions suddenly spike
- Compare actual vs predicted once kit_end is known (concept drift detection)
- Automated retraining triggers when drift exceeds PSI > 0.1

9.4 CI/CD & Orchestration

- Automated pipeline: data validation → feature engineering → training → evaluation → deployment
- Great Expectations / Pandera for data quality gates before training
- Airflow / Prefect DAGs for scheduled daily/weekly retraining
- Containerized inference (Docker + FastAPI / BentoML) for REST API serving

9.5 Production Architecture

Layer	Technology	Purpose
Data Ingestion	Airflow + dbt	Schedule ETL, transform raw factory data
Feature Store	Feast / Tecton	Compute and serve features with time-correctness
Training	MLflow + Optuna	Track experiments, tune hyperparameters, and version models
Serving	BentoML / FastAPI	Low-latency REST API for real-time predictions
Monitoring	Evidently AI	Data drift, model drift, performance decay
Alerting	PagerDuty / Slack	Automated alerts when thresholds are breached
Retraining	Airflow + CI/CD	Automated retrain on the drift trigger or schedule